ספר לימוד והדרכה מעשית

MAUI Learning Book

המדריך המעשי לפיתוח אפליקציות מובייל ודסקטופ ב-.NET MAUI

פרויקט מלווה: YouTube Hub (.NET MAUI & SQLite)

ארכיטקטורה: MVVM, Service Layer & Repository Pattern

טכנולוגיות: C# 12, .NET 10, XAML, SQLite-net-pcl, xUnit

שנת מהדורה: 2026

MAUI Learning Book — מדריך הלמידה לפיתוח .NET MAUI

ספר זה הוא יומן למידה מצטבר המלווה את הפרויקט, ומסביר כל שלב ומושג בשפה ברורה בעברית עם מונחי קוד מדויקים באנגלית.

שיעור 1 — התקנת SQLite והגדרת מודלים עם מפתחות, אינדקסים וקשרים

תאריך: 2026-08-19

1. מהו SQLite ולמה משתמשים בו ב-.NET MAUI?

- **SQLite** הוא בסיס נתונים רלציוני (טבלאי) קל-משקל, מהיר ועצמאי לחלוטין (Embedded / Serverless).
- בתוך אפליקציית מובייל ודסקטופ, מסד הנתונים נשמר כקובץ מקומי יחיד (למשל `YouTubeHub.db3`) בתיקיית האחסון המוגנת של האפליקציה (`FileSystem.AppDataDirectory`).
- הנתונים נשמרים לצמיתות (Persistence) גם לאחר סגירת האפליקציה או הפעלה מחדש של המכשיר.

2. התקנת חבילות ה-NuGet הנדרשות

כדי לעבוד עם SQLite ב-.NET MAUI בצורה אסינכרונית, אנו מתקינים שתי חבילות עיקריות ב-`MauiSkillTester.csproj`:

1. `sqlite-net-pcl` **(גרסה 1.9.172 ומעלה):** שכבת ה-ORM (Object-Relational Mapping) המאפשרת לנו לעבוד עם אובייקטים ב-C# במקום לכתוב פקודות SQL גולמיות.
2. `SQLitePCLRaw.bundle_green` **(גרסה 2.1.10 ומעלה):** ספריית ה-C הגולמית המבצעת את פעולות הקריאה/כתיבה למנוע ה-SQLite בכל מערכות ההפעלה (Android, iOS, Windows, macOS).

```xml
<ItemGroup>
    <PackageReference Include="sqlite-net-pcl" Version="1.9.172" />
    <PackageReference Include="SQLitePCLRaw.bundle_green" Version="2.1.10" />
</ItemGroup>
```

3. תכונות (Attributes) במודלים של SQLite

כאשר אנחנו מגדירים מחלקה ב-C#, אנחנו מקשטים אותה ואת תכונותיה ב-Attributes מתוך מרחב השמות `using SQLite;`:

תכונה (Attribute)	משמעות ותפקיד	דוגמה בקוד
`[Table("Name")]`	קובע את השם המדויק של הטבלה במסד הנתונים	`[Table("Videos")]`
`[PrimaryKey]`	מגדיר את העמודה כמזהה חד-ערכי וייחודי לכל שורה	`[PrimaryKey]`
`[AutoIncrement]`	מקצה מספר סידורי עולה אוטומטית (1, 2, 3...) בעת `Insert`	`[AutoIncrement]`
`[Indexed]`	יוצר עץ חיפוש מהיר המונע סריקה של כל הטבלה (Full Scan)	`[Indexed]`
`[NotNull]`	מבטיח שהשדה חייב להכיל ערך ולא יכול להיות `null`	`[NotNull]`
`[Ignore]`	מתעלם מהתכונה ולא יוצר עבורה עמודה בטבלה	`[Ignore]`

4. מידול קשר 1-לרבים (One-to-Many Relationship)

בפרויקט שלנו, **פלייליסט אחד** מכיל **הרבה סרטונים**.

- במודל `Playlist` (טבלת האב): יש מפתח ראשי `Id`.
- במודל `VideoItem` (טבלת הבן): יש שדה `PlaylistId` מקושט ב-`[Indexed]`, המצביע ל-`Id` של הפלייליסט שאליו הוא שייך.

CSHARP

```csharp
// Models/Playlist.cs
[Table("Playlists")]
public class Playlist
{
    [PrimaryKey, AutoIncrement]
    public int Id { get; set; }

    [Indexed, NotNull]
    public string Name { get; set; } = string.Empty;

    public string Description { get; set; } = string.Empty;
    public DateTime CreatedAt { get; set; } = DateTime.Now;
}

// Models/VideoItem.cs
[Table("Videos")]
public class VideoItem
{
    [PrimaryKey, AutoIncrement]
    public int Id { get; set; }

    [Indexed, NotNull]
    public int PlaylistId { get; set; } // מפתח זר

    [Indexed, NotNull]
    public string Title { get; set; } = string.Empty;

    public string Url { get; set; } = string.Empty;
    public string ChannelName { get; set; } = string.Empty;
    public int DurationMinutes { get; set; }

    [Indexed]
    public bool IsWatched { get; set; }

    public string Notes { get; set; } = string.Empty;
    public DateTime CreatedAt { get; set; } = DateTime.Now;
}
```

5. כיצד לוודא את התקינות

1. הרצת `dotnet build` לוודא שהחבילות יורדות בהצלחה והקוד מתקמפל ללא שגיאות.

6. שאלות חזרה ובקרה

1. מה ההבדל בין חיפוש שדה שיש עליו `[Indexed]` לבין שדה רגיל ללא אינדקס?
2. למה כדאי להשתמש ב-`sqlite-net-pcl` במקום לכתוב פקודות `CREATE TABLE` גולמיות במחרוזות SQL?

שיעור 2 — שירות בסיס הנתונים האסינכרוני (Database Service), תבנית אתחול עצל ו-Dependency Injection

תאריך: 2026-08-19

1. ארכיטקטורת שכבת השירות (Service Layer)

במקום לפתוח חיבור למסד הנתונים ישירות בתוך מסכי ה-UI או ה-ViewModels, אנו מרכזים את כל הגישה לנתונים בתוך מחלקת שירות ייעודית:

- `IDatabaseService`: ממשק (Interface) המגדיר מה השירות יודע לעשות (CRUD, סינון, מחיקה).
- `SQLiteDatabaseService`: המחלקה המממשת את הגישה בפועל באמצעות מנוע SQLite.

הפרדה זו נקראת **הפרדת תחומי אחריות (Separation of Concerns)** והיא מאפשרת לנו:

- להחליף מימוש בעתיד בקלות.
- לכתוב בדיקות יחידה (Unit Tests) עם שירות מדומה (Mock).

2. אתחול עצל (Lazy Initialization Pattern)

ב-.NET MAUI, אסור לבצע פעולות I/O או פתיחת חיבורי קבצים בתוך בנאי (`Constructor`). לכן, אנו משתמשים בתבנית אתחול עצל:

CSHARP

```
public async Task InitializeAsync()
{
    if (_connection ≠ null)
        return; // החיבור כבר אותחל, אין צורך באתחול נוסף

    _connection = new SQLiteAsyncConnection(_databasePath);
    await _connection.CreateTableAsync<Playlist>();
    await _connection.CreateTableAsync<VideoItem>();
    await SeedInitialDataAsync();
}
```

בכל מתודת CRUD, השורה הראשונה היא `await InitializeAsync();`, מה שמבטיח שהטבלאות והחיבור מוכנים בדיוק כשצריך אותם.

3. פעולות CRUD אסינכרוניות ושאילתות LINQ

פעולה	שיטת SQLite	דוגמה בקוד
Create (הוספה)	`InsertAsync`	`await _connection.InsertAsync(newPlaylist);`
Read All (שליפה כוללת)	`Table<T>().ToListAsync`	`await _connection.Table<Playlist>().OrderByDescending(p => p.CreatedAt).ToListAsync();`
Read by Filter (סינון לפי קשר)	`Where( ... ).ToListAsync`	`await _connection.Table<VideoItem>().Where(v => v.PlaylistId == playlistId).ToListAsync();`
Update (עדכון רשומה)	`UpdateAsync`	`await _connection.UpdateAsync(video);`
Delete (מחיקה)	`DeleteAsync`	`await _connection.DeleteAsync(video);`

4. מחיקה משורשרת (Cascade Delete)

כאשר מוחקים פלייליסט שלם, עלינו למחוק תחילה את כל הסרטונים השייכים לאותו פלייליסט, כדי שלא יישארו "סרטונים יתומים" במסד הנתונים:

CSHARP

```
public async Task<int> DeletePlaylistAsync(Playlist playlist)
{
    await InitializeAsync();

    // 1. מחיקת כל הסרטונים השייכים לפלייליסט
    await _connection.Table<VideoItem>()
                     .Where(v => v.PlaylistId == playlist.Id)
                     .DeleteAsync();

    // 2. מחיקת הפלייליסט עצמו
    return await _connection.DeleteAsync(playlist);
}
```

5. רישום ב-Dependency Injection כ-`Singleton`

בתוך `MauiProgram.cs`:

```csharp
builder.Services.AddSingleton<IDatabaseService, SQLiteDatabaseService>();
```

- `AddSingleton`: מייצר מופע אחד יחיד שמשותף לכל המסכים וה-ViewModels לאורך כל זמן ריצת האפליקציה. מונע פתיחה כפולה ונעילת קובץ ה-SQLite.

6. שאלות חזרה ובקרה

1. למה אסור ליצור את חיבור ה-SQLite בתוך הבנאי (`Constructor`) של ה-Service?
2. מה היתרון בביצוע `Cascade Delete` בעת מחיקת פלייליסט?

שיעור 3 — שכבת ה-ViewModel: ניהול מצב (State), רשימות חכמות (ObservableCollection), פקודות (Commands) ומשגר ה-YouTube (Launcher)

תאריך: 2026-08-19

1. מבנה שכבת ה-ViewModel

ה-ViewModel הוא המוח של המסך בארכיטקטורת **MVVM** (**M**odel-**V**iew-**V**iew**M**odel):

- `BaseViewModel`: מחלקת בסיס המממשת את `INotifyPropertyChanged` ומנהלת את תכונת `IsLoading` (עבור מחוון טעינה `ActivityIndicator`).
- `MainViewModel`: ה-ViewModel הראשי המקבל את `IDatabaseService` בבנאי (Constructor Injection), ומנהל את הפלייליסטים, הסרטונים, הסינונים והפקודות.

2. מנגנון עדכון המסך: `INotifyPropertyChanged`

כאשר ערך של תכונה משתנה ב-C#, עלינו ליידע את ה-XAML באמצעות קריאה ל-`OnPropertyChanged()`:

CSHARP

```
public Playlist? SelectedPlaylist
{
    get ⇒ _selectedPlaylist;
    set
    {
        if (_selectedPlaylist ≠ value)
        {
            _selectedPlaylist = value;
            OnPropertyChanged(); // מעדכן את המסך
            OnPropertyChanged(nameof(HasSelectedPlaylist));
            ((Command)AddVideoCommand).ChangeCanExecute(); // מעדכן האם כפתור ההוספה פעיל
        }
    }
}
```

3. `ObservableCollection<T>` — רשימה חיה

- בניגוד ל-`List<T>` רגיל שאינו שולח התראות למסך, `ObservableCollection<T>` מודיעה ל-XAML בכל פעם שנוסף פריט (`Add`), נמחק פריט (`Remove`) או מתנקה הרשימה (`Clear`).
- המסך ב-XAML מאזין לאירוע זה ומצייר את הפריטים מחדש באופן אוטומטי.

4. פקודות ב-MVVM (`ICommand` ו-`CanExecute`)

במקום לכתוב אירועי `Button_Clicked` בקובץ ה-Code-Behind של המסך, אנו מגדירים פקודות:

CSHARP

```
AddVideoCommand = new Command(
    execute: async () ⇒ await AddVideoAsync(),
    canExecute: () ⇒ SelectedPlaylist ≠ null &&
                     !string.IsNullOrWhiteSpace(NewVideoTitle) &&
                     !string.IsNullOrWhiteSpace(NewVideoUrl)
);
```

- `execute`: הקוד שירוץ בעת לחיצה על הכפתור.
- `canExecute`: תנאי לוגי הקובע האם הכפתור יהיה זמין (`Enabled`) או מנוטרל (אפור) ב-UI.

5. פתיחת קישור ל-YouTube עם `Launcher`

שירות ה-`Launcher` המובנה ב-.NET MAUI מאפשר לשגר קישורים חיצוניים ישירות למערכת ההפעלה:

```csharp
Uri uri = new Uri(video.Url);
await Launcher.Default.OpenAsync(uri);
```

מערכת ההפעלה (Android, iOS או Windows) תפתח אוטומטית את אפליקציית YouTube הרשמית או את הדפדפן להצגת הסרטון.

6. שאלות חזרה ובקרה

1. למה עדיף להשתמש ב-`Command` עם `CanExecute` במקום לכתוב קוד בלחיצת כפתור `Button_Clicked`?
2. כיצד `ObservableCollection` גורמת למסך להתעדכן מיד כשנוסף סרטון חדש?

שיעור 4 — עיצוב ממשק משתמש ב-XAML: תמיכה ב-RTL (עברית), Compiled Bindings, CollectionView ו-ActivityIndicator

---

תאריך: 2026-08-19

1. תמיכה בשפות מימין לשמאל (`FlowDirection="RightToLeft"`)

ב-.NET MAUI, תמיכה מלאה בשפות כמו עברית וערבית מתבצעת בקלות ע"י הגדרת תכונת ה-`FlowDirection` בתוך רכיב ה-`ContentPage`:

```xml
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:vm="clr-namespace:MauiSkillTester.ViewModels"
             x:Class="MauiSkillTester.MainPage"
             x:DataType="vm:MainViewModel"
             Title="{Binding Title}"
             FlowDirection="RightToLeft">
```

כאשר `FlowDirection="RightToLeft"`, כל הרכיבים (רשימות, כפתורים, תגיות, שדות קלט) משנים את סידורם אוטומטית מימין לשמאל, והטקסטים מיושרים לימין כנדרש בעברית.

2. קישורים מתוכפלים בזמן קומפילציה (`Compiled Bindings / x:DataType`)

כאשר אנחנו מציינים `x:DataType="vm:MainViewModel"` בראש דף ה-XAML:

- המהדר (Compiler) של .NET MAUI בודק בזמן הקומפילציה שכל שם תכונה ב-`Binding` אכן קיים ב-`MainViewModel`.
- אם יש טעות כתיב (למשל `Titlee` במקום `Title`), הקומפילציה תיכשל מיד (להיכשל בזמן ריצה (Runtime).
- הביצועים משתפרים משמעותית מכיוון שהמערכת מציבה קוד C# ישיר במקום להשתמש ב-Reflection איטי.

3. תצוגת אוספים מתקדמת עם `CollectionView`

הרכיב `CollectionView` הוא הרכיב המומלץ והמתקדם ביותר ב-.NET MAUI להצגת רשימות:

- `ItemsSource="{Binding Videos}"`: מקשר את הרשימה ל-`ObservableCollection<VideoItem>` ב-ViewModel.
- `DataTemplate`: קובע כיצד ייראה כל פריט ברשימה. אנו מציינים `x:DataType="models:VideoItem"` בתוך התבנית כדי לקבל Compiled Binding עבור שדות הסרטון הבודד.
- `EmptyView`: תצוגה מותאמת אישית המוצגת אוטומטית כאשר הרשימה ריקה (`Videos.Count == 0`).

XML

```xml
<CollectionView ItemsSource="{Binding Videos}">
    <CollectionView.EmptyView>
        <VerticalStackLayout Padding="30" HorizontalOptions="Center">
            <Label Text="📭 אין סרטונים להצגה בפלייליסט זה" TextColor="Gray" />
        </VerticalStackLayout>
    </CollectionView.EmptyView>
    <CollectionView.ItemTemplate>
        <DataTemplate x:DataType="models:VideoItem">
            <!-- עיצוב כרטיס סרטון -->
        </DataTemplate>
    </CollectionView.ItemTemplate>
</CollectionView>
```

4. מחוון טעינה אסינכרוני (`ActivityIndicator`)

אנו מציגים גלגל טעינה מסתובב בזמן שהנתונים נטענים מ-SQLite:

- `IsRunning="{Binding IsLoading}"`: מפעיל את האנימציה כש-`IsLoading == true`.
- `IsVisible="{Binding IsLoading}"`: מעלים את הרכיב כליל מהמסך כש-`IsLoading == false`.
- **ארכיטקטורה נקייה:** לעולם אין להשתמש ב-`Page.IsBusy` הישן, אלא לנהל את המצב ב-ViewModel.

5. שינוי עיצוב דינמי ב-XAML עם `DataTrigger`

במקום ליצור Convertor ייעודי, XAML מאפשר להגדיר טריגר דינמי המשנה את המראה של כפתור לפי ערך ב-Binding:

XML

```xml
<Button Text="טרם נצפה 👁️"
        BackgroundColor="#FFF3E0"
        TextColor="#E65100">
    <Button.Triggers>
        <DataTrigger TargetType="Button" Binding="{Binding IsWatched}" Value="True">
            <Setter Property="Text" Value="נצפה ✅" />
            <Setter Property="BackgroundColor" Value="#E8F5E9" />
            <Setter Property="TextColor" Value="#2E7D32" />
        </DataTrigger>
    </Button.Triggers>
</Button>
```

6. הזרקת תלויות (DI) של דף ה-MainPage

בתוך `MauiProgram.cs` רשמנו את הדף וה-ViewModel:

CSHARP

```csharp
builder.Services.AddSingleton<MainViewModel>();
builder.Services.AddSingleton<MainPage>();
```

ובקובץ הקוד של הדף (`MainPage.xaml.cs`), הבנאי מקבל את ה-ViewModel ומגדיר אותו כ-`BindingContext`:

CSHARP

```csharp
public MainPage(MainViewModel viewModel)
{
    InitializeComponent();
    BindingContext = viewModel;
}

protected override async void OnAppearing()
{
    base.OnAppearing();
    if (BindingContext is MainViewModel viewModel)
    {
        await viewModel.LoadDataAsync();
    }
}
```

7. שאלות חזרה ובקרה

1. מה היתרון הבולט של `x:DataType` (Compiled Bindings) ב-XAML?
2. כיצד התכונה `FlowDirection="RightToLeft"` משפיעה על סידור הרכיבים במסך?

שיעור 5 — בדיקות יחידה ב-xUnit, דיאגנוסטיקת קריסות בזמן ריצה ותבנית AAA

תאריך: 2026-08-19

1. מהן בדיקות יחידה (Unit Testing) ולמה הן חיוניות?

בדיקת יחידה היא קוד אוטומטי הבוחן רכיב לוגי בודד (כגון מתודת בסיס נתונים או פקודה ב-ViewModel) באופן מבודד לחלוטין מיתר חלקי המערכת.

- **גילוי באגים מוקדם:** מונע תקלות קריטיות בזמן ריצה אצל המשתמש.
- **הרצה מהירה במילי-שניות:** מאפשר להריץ עשרות בדיקות בשנייה בודדת עם `dotnet test`.

2. תבנית הבדיקה Standard AAA (Arrange, Act, Assert)

כל בדיקה ב-xUnit נכתבת במבנה של 3 שלבים ברורים:

CSHARP

```
[Fact]
public async Task SavePlaylistAsync_ShouldInsertNewPlaylist_AndAssignId()
{
    // 1. Arrange (הכנה) - הגדרת הרכיבים והנתונים
    var service = CreateService();
    var newPlaylist = new Playlist { Name = "סרטוני תכנות" };

    // 2. Act (ביצוע) - הפעלת המתודה הנבדקת
    int result = await service.SavePlaylistAsync(newPlaylist);

    // 3. Assert (אימות) - בדיקת התוצאה
    Assert.True(result > 0);
    Assert.True(newPlaylist.Id > 0);
}
```

3. דיאגנוסטיקה ותיקון קריסות בזמן ריצה (Runtime Crash Diagnostics)

במהלך הפיתוח, האפליקציה נתקלה בשתי סיבות מרכזיות לקריסה בזמן הניווט, אותן זיהינו ותיקנו:

1. `MissingMethodException` ב-**Shell Navigation:**
   - **הבעיה:** הרכיב `ShellContent` ב-`AppShell.xaml` ניסה ליצור מופע של `MainPage` דרך `DataTemplate`, שדרש בנאי ללא פרמטרים.
   - **הפתרון:** הוספת בנאי ברירת מחדל ב-`MainPage.xaml.cs`:

```csharp
public MainPage() : this(new MainViewModel(new SQLiteDatabaseService()))
{
}
```

2. `IndexOutOfRangeException` **בעת הצגת הודעות שגיאה:**
   - **הבעיה:** פנייה ל-`Application.Current.Windows[0]` בזמן אתחול לפני שהחלון הראשון נוצר.
   - **הפתרון:** כתיבת עוזר בטוח `ShowAlertAsync` המשתמש ב-`Shell.Current.DisplayAlertAsync` עם בדיקת תקינות.

4. גילוי באג מורכב בעזרת בדיקות היחידה (Race Condition / State Bug)

כאשר הרצנו את בדיקות ה-xUnit, בדיקת טעינת הנתונים נכשלה מכיוון שהתכונה `IsLoading = true` חסמה את טעינת הסרטונים בעת עדכון הפלייליסט הנבחר. בעזרת הבדיקה תיקנו את הלוגיקה ב-`MainViewModel.cs` כך שכל הסרטונים נטענים בצורה אסינכרונית בטוחה!

5. שאלות חזרה ובקרה

1. מה התפקיד של כל אחד משלושת השלבים בתבנית AAA?
2. כיצד בדיקות יחידה ב-xUnit עזרו לנו לגלות באגים בקוד עוד לפני שהרצנו את המכשיר הנייד?

פרק 6: תצורת SQLite רשמית של Microsoft, ניקוי התנגשויות ו-Dependency Injection

בפרק זה יישרנו את הפרויקט ב-100% מול התקן והתיעוד הרשמי של מיקרוסופט (Microsoft Learn: .NET MAUI Local Databases with SQLite) ופתרנו קריסות מערכת בעת עליית האפליקציה.

1. ניקוי התנגשויות ספריות נייטיביות (Native Provider Bundles Conflict)

שורש הבעיה שגרמה לקריסה:

בקובץ הפרויקט `MauiSkillTester.csproj` הוגדרו מספר חבילות NuGet של ספקי SQLite שהתנגשו זו בזו:

- `SQLitePCLRaw.bundle_green`
- `SQLitePCLRaw.bundle_e_sqlite3`
- `SQLitePCLRaw.lib.e_sqlite3.android`

בזמן עליית האפליקציה, כאשר נקראה המתודה `Batteries_V2.Init()`, המערכת ניסתה לטעון שני ספקים מקבילים עם P/Invoke סותרים, מה שגרם לקריסה מיידית של האפליקציה (Native Crash).

הפתרון:

הסרת כל החבילות העודפות והשארת התקן המומלץ והנקי של מיקרוסופט בלבד:

- `sqlite-net-pcl` (גרסה 1.11.285 ומעלה)
- `SQLitePCLRaw.bundle_green` (גרסה 2.1.11 ומעלה)

2. מחלקת הקבועים המרכזית (`Constants.cs`)

מיקרוסופט ממליצה לרכז את כל נתוני התצורה של מסד הנתונים בקובץ סטטי מרכזי:

CSHARP

```
using SQLite;

namespace MauiSkillTester;

public static class Constants
{
    public const string DatabaseFilename = "YouTubeHub.db3";

    public const SQLiteOpenFlags Flags =
        SQLiteOpenFlags.ReadWrite |
        SQLiteOpenFlags.Create |
        SQLiteOpenFlags.SharedCache;

    public static string DatabasePath ⇒
        Path.Combine(FileSystem.AppDataDirectory, DatabaseFilename);
}
```

הסבר על הדגלים (`SQLiteOpenFlags`):

1. `ReadWrite`: מאפשר גם קריאת נתונים (SELECT) וגם כתיבה/עדכון (INSERT/UPDATE/DELETE).
2. `Create`: יוצר את קובץ ה-`.db3` באופן אוטומטי אם הוא אינו קיים במכשיר.
3. `SharedCache`: מאפשר גישה מרובת תהליכונים (Multi-threaded access) ושיתוף זיכרון מטמון למניעת נעילת קבצים.

3. אתחול מודולרי ב-`SQLiteDatabaseService.cs`

עדכנו את בנאי השירות כך שייעשה שימוש ב-`Constants.DatabasePath` וב-`Constants.Flags`, עם תמיכה באתחול עצל (Lazy Initialization) ויכולת העברת נתיב מותאם לצורכי בדיקות יחידה:

CSHARP

```csharp
public SQLiteDatabaseService() : this(Constants.DatabasePath, Constants.Flags)
{
}

public SQLiteDatabaseService(string databasePath, SQLiteOpenFlags flags = Constants.Flags)
{
    _databasePath = databasePath;
    _flags = flags;
}

public async Task InitializeAsync()
{
    if (_connection ≠ null)
        return;

    SQLitePCL.Batteries_V2.Init();
    _connection = new SQLiteAsyncConnection(_databasePath, _flags);

    await _connection.CreateTableAsync<Playlist>();
    await _connection.CreateTableAsync<VideoItem>();
    await SeedInitialDataAsync();
}
```

4. רישום שירותים במכולת התלויות ב- `MauiProgram.cs` (Dependency Injection)

כדי להבטיח שכל חלקי האפליקציה חולקים מופע יחיד של שירות הנתונים (Singleton), רשמנו את השירותים, את ה-ViewModel ואת ה-Page ב- `MauiProgram.cs` :

CSHARP

```csharp
// רישום שכבת הנתונים, ה-ViewModel וה-View במכולת התלויות (DI Container)
builder.Services.AddSingleton<IDatabaseService, SQLiteDatabaseService>();
builder.Services.AddSingleton<MainViewModel>();
builder.Services.AddSingleton<MainPage>();
```

5. שאלות חזרה ובקרה

1. מדוע התנגשות בין ספריות `SQLitePCLRaw` גורמת לקריסה של האפליקציה?
2. מה היתרון של שימוש ב- `Constants.Flags` הכולל `SharedCache` בעת פתיחת חיבור מסד הנתונים?
3. כיצד שיטת ה-Singleton ב- `MauiProgram.cs` מונעת שגיאות של Database Locked?

פרק 7: שאילתות אגרגציה ב-SQLite, פונקציית COALESCE ואנוטציית [Ignore]

תאריך: 2026-08-20

בפרק זה למדנו כיצד להפיק סטטיסטיקות וסיכומי נתונים בעלי ביצועים גבוהים ישירות ממנוע ה-SQLite, להגן מפני ערכי `NULL`, ולהגדיר תכונות עזר לתצוגה ב-XAML מבלי לפגוע במבנה הטבלה.

1. למה לבצע אגרגציות ב-SQL ולא ב-C#?

כאשר באפליקציה יש מאות או אלפי רשומות, משיכת כל הרשומות ל-C# וספירתן בזיכרון (`videos.Count()`) גוזלת זיכרון RAM ומשאבי מעבד מיותרים. שאילתות אגרגציה ב-SQL (`COUNT`, `SUM`, `AVG`) מבוצעות ישירות על קובץ בסיס הנתונים ומחזירות **ערך מספרי בודד** במיקרו-שניות.

2. ביצוע שאילתות סקלריות עם `ExecuteScalarAsync<T>`

כאשר שאילתת SQL מחזירה ערך בודד בלבד (כמו מספר `int`), נשתמש במתודה `ExecuteScalarAsync<T>`:

CSHARP

```
public async Task<PlaylistStats> GetPlaylistStatsAsync(int playlistId)
{
    await InitializeAsync();

    // 1. ספירת סך כל הסרטונים בפלייליסט
    int totalVideos = await _connection!.ExecuteScalarAsync<int>(
        "SELECT COUNT(*) FROM Videos WHERE PlaylistId = ?", playlistId);

    // 2. ספירת סרטונים שנצפו
    int watchedVideos = await _connection!.ExecuteScalarAsync<int>(
        "SELECT COUNT(*) FROM Videos WHERE PlaylistId = ? AND IsWatched = 1", playlistId);

    // 3. סכימת סך כל דקות הצפייה
    int totalDuration = await _connection!.ExecuteScalarAsync<int>(
        "SELECT COALESCE(SUM(DurationMinutes), 0) FROM Videos WHERE PlaylistId = ?",
playlistId);

    return new PlaylistStats
    {
        PlaylistId = playlistId,
        TotalVideos = totalVideos,
        WatchedVideos = watchedVideos,
        TotalDurationMinutes = totalDuration
    };
}
```

3. פונקציית `COALESCE` להגנה מפני ערכי `NULL`

כאשר מבצעים `SUM(DurationMinutes)` על פלייליסט שאין בו סרטונים כלל, SQL מחזיר ערך `NULL` (ולא `0`). הפונקציה `COALESCE(ערך, ערך_ברירת_מחדל)` מבטיחה שאם התוצאה היא `NULL`, יוחזר הערך `0` ובכך מונעת חריגות וקריסות המרה ב-C#.

4. מודל DTO ייעודי לסטטיסטיקות (`PlaylistStats.cs`)

יצרנו מחלקה קלה (Data Transfer Object) המרכזת את תוצאות השאילתות ומכילה תכונות מחושבות לתצוגה:

CSHARP

```
public class PlaylistStats
{
    public int PlaylistId { get; set; }
    public int TotalVideos { get; set; }
    public int WatchedVideos { get; set; }
    public int UnwatchedVideos => TotalVideos - WatchedVideos;
    public int TotalDurationMinutes { get; set; }

    public double CompletionPercentage =>
        TotalVideos > 0 ? Math.Round((double)WatchedVideos / TotalVideos * 100, 1) : 0.0;

    public string FormattedTotalDuration
    {
        get
        {
            if (TotalDurationMinutes < 60)
                return $"{TotalDurationMinutes} דקות";
            int hours = TotalDurationMinutes / 60;
            int mins = TotalDurationMinutes % 60;
            return mins > 0 ? $"{hours} שעות ו-{mins} דק'" : $"{hours} שעות";
        }
    }
}
```

5. אנוטציית `[SQLite.Ignore]` ב-Models

ב-`sqlite-net-pcl`, כל תכונה ציבורית (`public Property`) במחלקה נחשבת כעמודה בטבלה. כאשר אנו רוצים להוסיף תכונת עזר מחושבת עבור ה-XAML (כמו `FormattedDuration`), אנו מוסיפים את האנוטציה `[Ignore]`:

CSHARP

```
[Ignore]
public string FormattedDuration => DurationMinutes > 0 ? $"{DurationMinutes} לא צוין" : "'דק";

[Ignore]
public string WatchedStatusText => IsWatched ? "טרם נצפה" : "✓ נצפה ⌛";
```

באמצעות `[Ignore]`, מנוע ה-SQLite מתעלם לחלוטין מתכונות אלו בעת יצירת הטבלה או שמירת רשומות.

6. תצוגת דשבורד סטטיסטי ב-XAML (חיבור ה-DTO לממשק)

ב-`MainPage.xaml`, הוספנו סרגל כרטיסי מידע מודרני המוצג רק כאשר נבחר פלייליסט עם נתונים (`IsVisible="{Binding HasStats}"`). הממשק משתמש ב-`Border` עם פינות מעוגלות (`RoundRectangle 8`), התאמה אוטומטית למצב כהה ובהיר (`AppThemeBinding`), וחלוקה לשלוש עמודות שוות:



XML

```xml
<Border StrokeShape="RoundRectangle 10"
        Padding="14"
        BackgroundColor="{AppThemeBinding Light=#EDE7F6, Dark=#311B92}"
        IsVisible="{Binding HasStats}">
    <VerticalStackLayout Spacing="8">
        <Label Text="📊 סיכום סטטיסטי (SQLite Aggregations)"
               FontSize="15"
               FontAttributes="Bold"
               TextColor="{AppThemeBinding Light=#4A148C, Dark=#E1BEE7}" />

        <Grid ColumnDefinitions="*, *, *" ColumnSpacing="8">
            <!-- כרטיס 1: סה"כ סרטונים -->
            <Border StrokeShape="RoundRectangle 8"
                    BackgroundColor="{AppThemeBinding Light=White, Dark=#4527A0}"
                    Padding="8">
                <VerticalStackLayout HorizontalOptions="Center" Spacing="2">
                    <Label Text="🎬 סה&amp;quot;כ"
                           FontSize="11"
                           TextColor="Gray"
                           HorizontalOptions="Center" />
                    <Label Text="{Binding CurrentPlaylistStats.TotalVideos}"
                           FontSize="18"
                           FontAttributes="Bold"
                           HorizontalOptions="Center" />
                </VerticalStackLayout>
            </Border>

            <!-- כרטיס 2: סרטונים שנצפו ואחוז השלמה -->
            <Border StrokeShape="RoundRectangle 8"
                    BackgroundColor="{AppThemeBinding Light=White, Dark=#4527A0}"
                    Padding="8">
                <VerticalStackLayout HorizontalOptions="Center" Spacing="2">
                    <Label Text="✅ נצפו"
                           FontSize="11"
                           TextColor="Gray"
                           HorizontalOptions="Center" />
                    <Label Text="{Binding CurrentPlaylistStats.WatchedVideos, StringFormat='{0}
סרטונים'}"
                           FontSize="13"
                           FontAttributes="Bold"
                           HorizontalOptions="Center" />
                    <Label Text="{Binding CurrentPlaylistStats.CompletionPercentage,
StringFormat='({0}%)'}"
                           FontSize="11"
                           TextColor="#2E7D32"
                           HorizontalOptions="Center" />
                </VerticalStackLayout>
            </Border>

            <!-- כרטיס 3: סך דקות צפייה -->
            <Border StrokeShape="RoundRectangle 8"
                    BackgroundColor="{AppThemeBinding Light=White, Dark=#4527A0}"
                    Padding="8">
                <VerticalStackLayout HorizontalOptions="Center" Spacing="2">
                    <Label Text="⏱️ סה&amp;quot;כ זמן"
                           FontSize="11"
                           TextColor="Gray"
```

```
                        HorizontalOptions="Center" />
                <Label Text="{Binding CurrentPlaylistStats.FormattedTotalDuration}"
                       FontSize="13"
                       FontAttributes="Bold"
                       HorizontalOptions="Center" />
            </VerticalStackLayout>
        </Border>
    </Grid>
</VerticalStackLayout>
</Border>
```

7. אימות מלא בבדיקות יחידה (xUnit)

כתבנו בדיקות יחידה ייעודיות ב-`DatabaseServiceTests.cs` וב-`MainViewModelTests.cs` שמאמתות:

1. חישוב נכון ומדויק של אחוזי השלמה, סה"כ זמנים וספירת פריטים שנצפו.
2. טיפול תקין בפלייליסט ריק (החזרת 0 ואחוז 0 ללא שגיאות).
3. כל 15 בדיקות היחידה עוברות בהצלחה מלאה (PASS 100%)!

8. שאלות חזרה, הבנה ואתגר חשיבה

⚖️ דילמת תכנון: מתי לבצע אגרגציות ב-SQL ומתי ב-C#?

נניח שבאפליקציה יש 1,000 סרטונים.

- **דרך א':** לשלוף את כולם לזיכרון ה-RAM ב-C# ולחשב `Videos.Count()`.
- **דרך ב':** לשלוח שאילתת `SELECT COUNT(*)` ישירות ל-SQLite. איזו דרך תצרוך פחות סוללה וזיכרון במכשיר נייד, ומדוע?

🧠 שאלות הבנה מעמיקות עם רמזים

1. **מה ההבדל המעשי בין `QueryAsync<T>` לבין `ExecuteScalarAsync<T>`?**

> 💡 **רמז לחשיבה:** חשבו על הערך המוחזר — האם מדובר באוסף של שורות שלמות (`List<T>`) או בערך סקלרי בודד (כמו מספר שלם יחיד)?

2. **מדוע חיוני להשתמש בפונקציה `COALESCE(SUM( ... ), 0)` כשמחשבים זמן כולל?**

> 💡 **רמז לחשיבה:** מה מחזירה שאילתת `SUM` כאשר הטבלה ריקה לגמרי ואין בה אף שורה?

3. **מה היה קורה למבנה הטבלה במסד הנתונים אילו שכחנו להוסיף `[Ignore]` מעל התכונה `FormattedDuration`?**

> 💡 **רמז לחשיבה:** `sqlite-net-pcl` מנסה להפוך כל `public property` לעמודה ב-SQL. האם ניתן לשמור `string` מחושב בטבלה ללא שדה תואם?

פרק 8: שאילתות פרמטריות דינמיות ב-SQL, מנגנון LIKE עם ESCAPE ומיון חכם

תאריך: 2026-08-20

בפרק זה שדרגנו את מנגנון החיפוש והסינון של האפליקציה: במקום למשוך את כל הנתונים לזיכרון ה-RAM ולסנן ב-C# באמצעות LINQ, העברנו את העבודה הכבדה ישירות למנוע ה-SQLite באמצעות **שאילתות פרמטריות דינמיות**.

1. למה סינון ב-SQL עדיף על סינון ב-C# (In-Memory)?

- **חיסכון בזיכרון:** כאשר יש אלפי סרטונים, טעינת כולם לזיכרון והפעלת `Where` ב-C# גורמת לתקיעות (Lag) וצריכת סוללה.
- **מהירות גבוהה:** מנוע ה-SQLite משתמש באינדקסים שהגדרנו (`[Indexed]`) ומחזיר רק את השורות הרלוונטיות ביותר.

2. בניית שאילתה דינמית עם פרמטריזציה (`?`)

במתודה `GetFilteredVideosAsync` אנו בונים את השאילתה בצורה מודולרית תוך שמירה על עקרון הברזל של אבטחת מידע — **לעולם לא לשרשר קלט משתמש ישירות**:

```csharp
var sqlBuilder = new StringBuilder("SELECT * FROM Videos WHERE PlaylistId = ?");
var parameters = new List<object> { playlistId };

if (onlyUnwatched)
{
    sqlBuilder.Append(" AND IsWatched = 0");
}
```

3. מנגנון ה-`LIKE` ותו המילוט `ESCAPE '/'`

כאשר המשתמש מחפש טקסט, אנו רוצים למצוא התאמה גם אם המילה נמצאת באמצע המשפט. לכן אנו עוטפים את הטקסט בסימני אחוז `$"%{escaped}%"`.

איך מחפשים סימן אחוז או קו תחתי מילולי מבלי ש-SQL יפרש אותם כפקודות?

משתמשים בהוראת `ESCAPE '/'` ומחליפים תווים מיוחדים לפני שליחתם:

CSHARP

```
if (!string.IsNullOrWhiteSpace(searchQuery))
{
    // מילוט תווים מיוחדים
    string escaped = searchQuery.Trim()
        .Replace("/", "//")
        .Replace("%", "/%")
        .Replace("_", "/_");

    string pattern = $"%{escaped}%";

    sqlBuilder.Append(" AND (Title LIKE ? ESCAPE '/' OR ChannelName LIKE ? ESCAPE '/' OR Notes
LIKE ? ESCAPE '/')");
    parameters.Add(pattern);
    parameters.Add(pattern);
    parameters.Add(pattern);
}
```

4. מיון דינמי (`ORDER BY`) ו-`COLLATE NOCASE`

הגדרנו `enum VideoSortOption` המאפשר למשתמש לבחור כיצד למיין את התוצאות:

CSHARP

```
string orderByClause = sortBy switch
{
    VideoSortOption.ShortestFirst ⇒ " ORDER BY DurationMinutes ASC, CreatedAt DESC",
    VideoSortOption.LongestFirst ⇒ " ORDER BY DurationMinutes DESC, CreatedAt DESC",
    VideoSortOption.Alphabetical ⇒ " ORDER BY Title COLLATE NOCASE ASC",
    _ ⇒ " ORDER BY CreatedAt DESC"
};

sqlBuilder.Append(orderByClause);
```

שים לב: ההוראה `COLLATE NOCASE` מבטיחה שסרטון שמתחיל באות `a` קטנה יופיע לפני סרטון שמתחיל באות `Z` גדולה, למיין א-ב טבעי ונכון.

5. חוויית משתמש בזמן אמת ב-MVVM

ב-`MainViewModel.cs`, הגדרנו שכל שינוי ב-`SearchQuery`, ב-`ShowOnlyUnwatched` או ב-`SelectedSortOptionItem` יפעיל מיד את הטעינה מחדש ממסד הנתונים:

CSHARP

```
public string SearchQuery
{
    get ⇒ _searchQuery;
    set
    {
        if (_searchQuery ≠ value)
        {
            _searchQuery = value;
            OnPropertyChanged();
            _ = LoadVideosForSelectedPlaylistAsync();
        }
    }
}
```

6. אימות בבדיקות יחידה (xUnit)

כתבנו בדיקות יחידה חדשות המאמתות:

1. סינון לפי ערוץ וכותרת ב-SQL.
2. סינון סרטונים שלא נצפו.
3. מיון עולה ויורד לפי משך הזמן.
4. חיפוש מילולי של `"100%"` בעזרת מנגנון ה-ESCAPE.
5. **כל 19 בדיקות היחידה עברו בהצלחה מלאה (100% PASS)!**

7. שאלות חזרה ובקרה

1. מה תפקידה של ההוראה `ESCAPE '/'` בתוך שאילתת `LIKE`?
2. למה השתמשנו ב-`COLLATE NOCASE` במיון לפי שם הסרטון?
3. כיצד השימוש בסימני שאלה `?` מגן על האפליקציה מפני מתקפות SQL Injection?

נספח א' — סדרת מדריכי ממשק משתמש בשיטת הבלוקים המודולרית

חלק זה מרכז את מדריכי הממשק המודולריים של האפליקציה. כל מדריך כולל דיאגרמת בלוקים חזותית, צילומי מסך ממוקדים של כל בלוק, פירוק לגריד ולרכיבי XAML עצמאיים, טיפים מעמיקים לעיצוב פרימיום ב-.NET MAUI, וחיבור מלא ל-MVVM.

📚 אינדקס המדריכים:

- ספריית מדריכי הממשק והעקרונות

- 🎬 **[מדריך פיתוח MainPage — מרכז למידה ומעקב YouTube]**:
  - פירוק מודולרי ל-5 בלוקים חזותיים (ניהול פלייליסט, אגרגציות SQLite, טופס הוספה, סרגל סינון ומיון, ורשימת CollectionView).
  - טכניקות מפתח: `DataTrigger`, `RelativeSource AncestorType`, `AppThemeBinding`, ופקודות `ICommand.CanExecute`.
- *מדריכי מסכים נוספים יתווספו כאן במקביל להרחבת הפרויקט.*

נספח ב' — מקורות השראה, תיעוד וביבליוגרפיה

ספר למידה זה ומדריכי הממשק שבו פותחו בהשראת מתודולוגיית ההוראה המודולרית **The Block Strategy** של **Leomaris Reyes** (Microsoft MVP, בלוגרית Telerik ויוצרת AskXammy.com), המנגישה עיצובי ממשק מובייל מורכבים ב-.NET MAUI בצורה הדרגתית ומובנית.

תיעוד רשמי ומקורות להעמקה:

- **Microsoft Learn (.NET MAUI):** תיעוד רשמי של ארכיטקטורת האפליקציה, מחזור החיים ובקרי הממשק (learn.microsoft.com/dotnet/maui).
- **Data Binding & MVVM:** דפוסי חיבור נתונים, פקודות ICommand ו-Compiled Bindings (learn.microsoft.com/dotnet/maui/fundamentals/data-binding).
- **SQLite-net-pcl & SQLitePCLRaw:** אחסון מקומי מוצפן, אינדקסים, קשרים טבלאיים ושאילתות מהירות במובייל.
- **xUnit Testing Framework:** בדיקות יחידה לתשתיות נתונים ומודלי תצוגה בתבנית Standard AAA.

פרק הרחבה — מדריך פיתוח מפורט: MainPage (שיטת הבלוקים המודולרית)

שיטת הבלוקים המודולרית לבניית ממשק משתמש ב-.NET MAUI.

🎯 1. מבוא והשראה עיצובית: בונים מסך ניהול תוכן מתקדם ב-MAUI

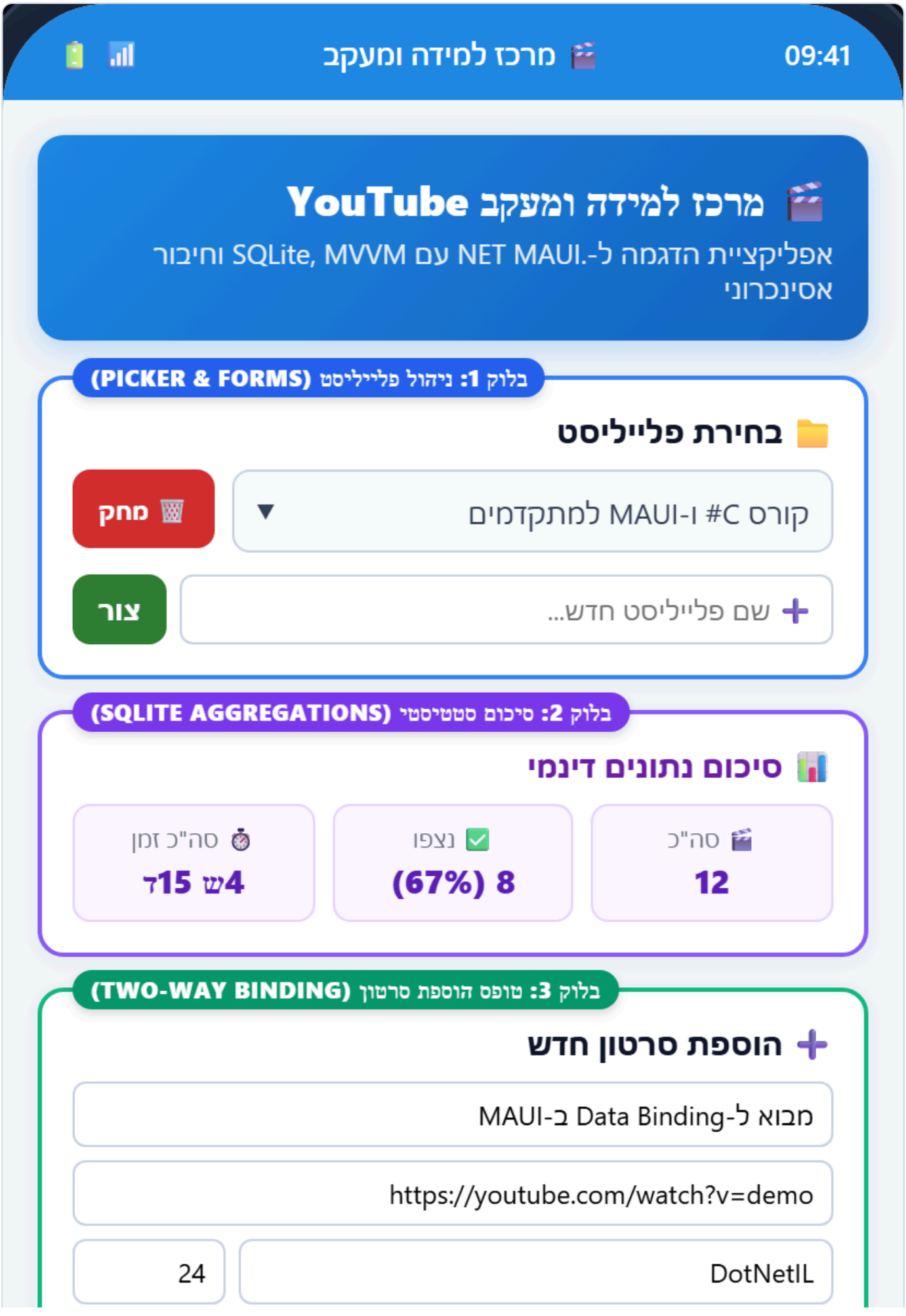
שמתם לב פעם באפליקציות המובילות — כמו Spotify, YouTube או Notion — איך מסכים מורכבים שמכילים עשרות פקדים, טפסים, נתונים סטטיסטיים ורשימות נראים מסודרים, ברורים ומזמינים? במבט ראשון, מסך כזה עלול להיראות מאיים לתכנות: יש בו בחירת קטגוריה, כרטיסי סיכום מחושבים, טופסי קלט, מנוע חיפוש וסינון, ורשימה דינמית של כרטיסים אינטראקטיביים.

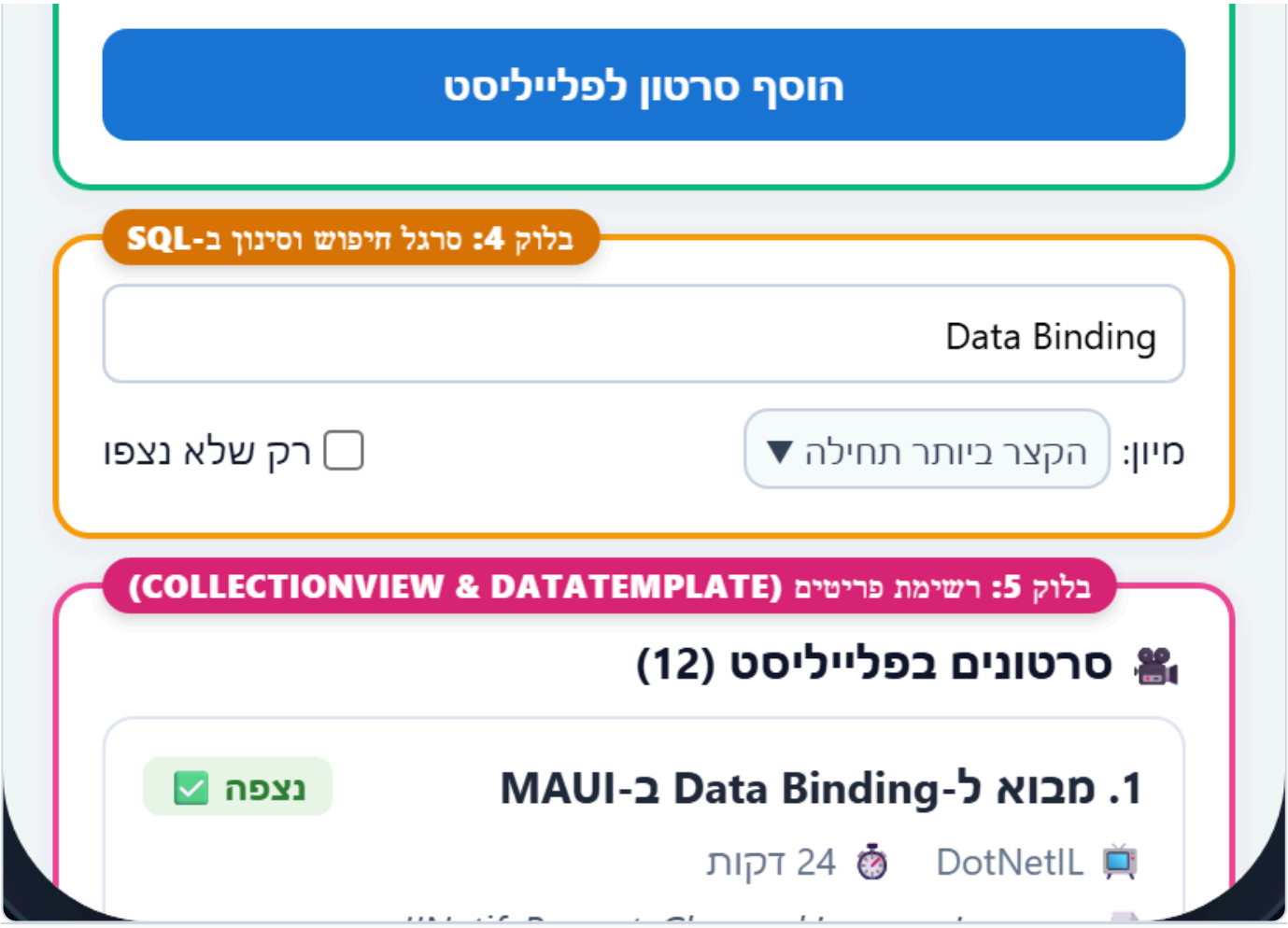
הסוד של מפתחי מובייל מקצועיים הוא **לא לכתוב קובץ XAML ענק בבת אחת**, אלא לפרק את הממשק ל**בלוקים פונקציונליים עצמאיים**. במדריך זה נלמד לבנות את מסך ה-`MainPage` של פרויקט **YouTube Hub** צעד-אחר-צעד, באופן מודולרי ונקי, תוך

יישום שיטת הבלוקים המודולרית וחיבור מלא ל-MVVM ולמסד נתונים מקומי ב-SQLite!

🧱 מפת הבלוקים של המסך (Visual Block Architecture)

להלן מפת היעד החזותית של המסך, כפי שחולקה ל-5 בלוקים מרכזיים:


09:41
מרכז למידה ומעקב
מרכז למידה ומעקב YouTube
אפליקציית הדגמה ל-.NET MAUI עם MVVM, SQLite וחיבור אסינכרוני
בלוק 1: ניהול פלייליסט (PICKER & FORMS)
בחירת פלייליסט
קורס C# ו-MAUI למתקדמים
מחק
שם פלייליסט חדש...
צור
בלוק 2: סיכום סטטיסטי (SQLITE AGGREGATIONS)
סיכום נתונים דינמי
סה"כ
12
נצפו
8 (67%)
סה"כ זמן
4ש 15ד
בלוק 3: טופס הוספת סרטון (TWO-WAY BINDING)
הוספת סרטון חדש
מבוא ל-Data Binding ב-MAUI
https://youtube.com/watch?v=demo
DotNetIL
24



🧭 2. איך המדריך הזה בנוי? (The Meta-Frame)

כדי שתוכלו להבין כל שורת קוד ולשלוט בממשק ב-100%, נפעל לפי 3 עקרונות פדגוגיים מנחים:

- **דיאגרמה חזותית מפורקת (Visual Diagram):** חילקנו את המסך לחמישה בלוקים צבעוניים מוגדרים. בכל שלב נתמקד בבלוק אחד בלבד.
- **קוד מודולרי עם נקודות עוגן (Modular Code with Placeholders):** נתחיל עם שלד הגריד והערות מקום (`<!-- כאן -->`, `<!-- יבוא הבלוק הבא -->`), ונרכיב כל רכיב בנפרד.
- **התקדמות הדרגתית מלווה בצילומי מסך ממוקדים:** לכל בלוק מוצגת תמונת תוצאה מתוך ההדמיה, לצד פינות העמקה (`Deep-Dive Callouts`) על טכניקות מפתח.

🔧 3. הכנת סביבת העבודה ומרחבי שמות (Namespaces)

לפני שמתחילים בעיצוב ה-XAML, נוודא שראש הקובץ `MainPage.xaml` כולל את מרחבי השמות (Namespaces) הנדרשים לקישור ה-ViewModel והמודלים, וכן את הגדרת כיוון המסך מימין-לשמאל:

XML

```xml
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:vm="clr-namespace:MauiSkillTester.ViewModels"
             xmlns:models="clr-namespace:MauiSkillTester.Models"
             x:Class="MauiSkillTester.MainPage"
             x:DataType="vm:MainViewModel"
             Title="{Binding Title}"
             FlowDirection="RightToLeft">

    <!-- כאן יבוא שלד הגריד הראשי -->

</ContentPage>
```

💡 **מדוע הגדרנו** `x:DataType="vm:MainViewModel"`**?**

זהו מנגנון **Compiled Bindings** של .NET MAUI. הוא מאפשר לקומפיילר לבדוק בזמן הידור שכל שדה ופקודה קיימים באמת ב-ViewModel, מונע שגיאות כתיב חבויות, ומשפר את ביצועי האפליקציה בריצה!

📐 4. אסטרטגיית הבלוקים (The Block Strategy)

שלב 4.0: שלד הגריד הראשי ומחוון הטעינה (Grid Backbone & Loading)

עמוד השדרה של המסך מורכב מגריד בן 2 שורות:

1. שורה 0 (`Auto`): מיועדת למחוון הטעינה (`ActivityIndicator`).
2. שורה 1 (`*`): אזור הגלילה הראשי (`ScrollView`) המכיל את כל תוכן המסך.

```xml
<Grid RowDefinitions="Auto, *">

    <!-- מחוון טעינה אסינכרוני (ActivityIndicator) -->
    <ActivityIndicator Grid.Row="0"
                       IsRunning="{Binding IsLoading}"
                       IsVisible="{Binding IsLoading}"
                       Color="{StaticResource Primary}"
                       HeightRequest="40"
                       HorizontalOptions="Center"
                       VerticalOptions="Center" />

    <!-- מיכל הגלילה הראשי -->
    <ScrollView Grid.Row="1">
        <VerticalStackLayout Padding="16" Spacing="20">
            <!-- כותרת ראשית (Hero Header) -->
            <!-- בלוק 1: ניהול ובחירת פלייליסט -->
            <!-- בלוק 2: סיכום סטטיסטי של הפלייליסט -->
            <!-- בלוק 3: טופס הוספת סרטון -->
            <!-- בלוק 4: סרגל חיפוש, סינון ומיון ב-SQL -->
            <!-- בלוק 5: רשימת סרטונים דינמית -->
        </VerticalStackLayout>
    </ScrollView>

</Grid>
```

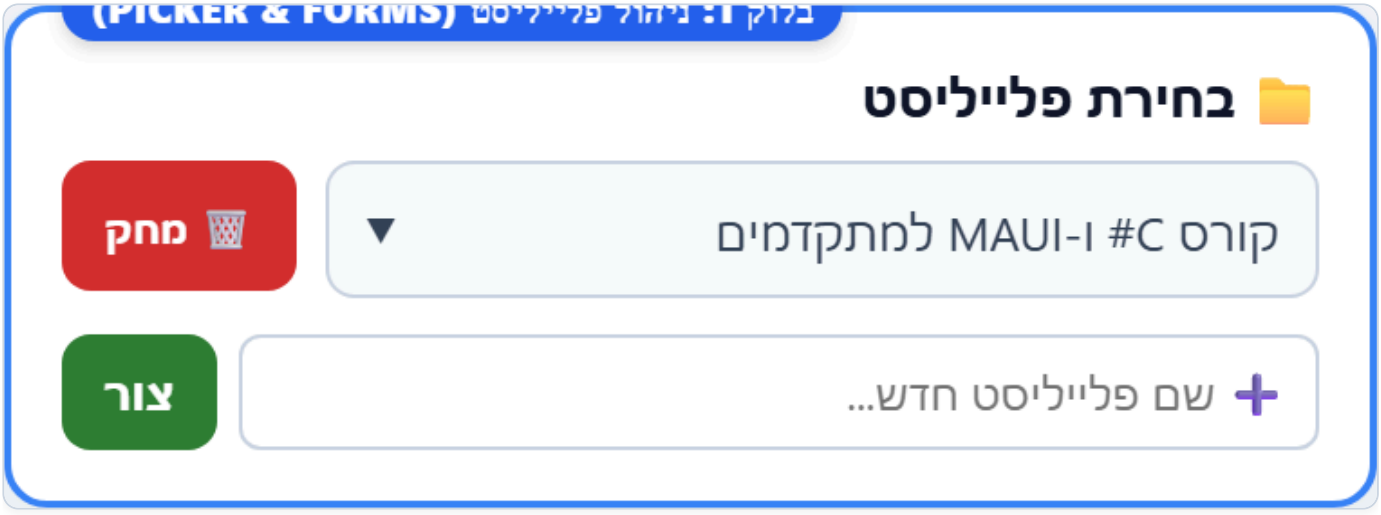
שלב 4.1: כותרת עליונה ובאנר ראשי (Hero Banner)



כרטיס כותרת צבעוני בעל פינות מעוגלות המעניק רושם ראשוני מקצועי:

```xml
<!-- כותרת ראשית ומעוצבת -->
<Border StrokeShape="RoundRectangle 12"
        BackgroundColor="#1E88E5"
        Padding="16"
        StrokeThickness="0">
    <VerticalStackLayout Spacing="6">
        <Label Text="🎬 מרכז למידה ומעקב YouTube"
               FontSize="22"
               FontAttributes="Bold"
               TextColor="White"
               HorizontalOptions="Start" />
        <Label Text="אפליקציית הדגמה ל– NET MAUI עם SQLite, MVVM וחיבור אסינכרוני"
               FontSize="14"
               TextColor="#E3F2FD"
               HorizontalOptions="Start" />
    </VerticalStackLayout>
</Border>
```

שלב 4.2: בלוק 1 — ניהול ובחירת פלייליסט (Playlist Management)



בלוק זה מאפשר למשתמש לבחור פלייליסט פעיל מתוך בורר (`Picker`), למחוק פלייליסט קיים, או ליצור פלייליסט חדש בטופס מהיר.

XML

```xml
<!-- חלק 1: ניהול ובחירת פלייליסט -->
<Border StrokeShape="RoundRectangle 10"
        Padding="16"
        BackgroundColor="{AppThemeBinding Light=#F5F5F5, Dark=#212121}">
    <VerticalStackLayout Spacing="12">
        <Label Text="📁 בחירת פלייליסט" FontSize="18" FontAttributes="Bold" />

        <Grid ColumnDefinitions="*, Auto" ColumnSpacing="10">
            <Picker Grid.Column="0"
                    Title="בחר פלייליסט מהרשימה..."
                    ItemsSource="{Binding Playlists}"
                    ItemDisplayBinding="{Binding Name}"
                    SelectedItem="{Binding SelectedPlaylist}" />

            <Button Grid.Column="1"
                    Text="🗑 מחק פלייליסט"
                    Command="{Binding DeletePlaylistCommand}"
                    CommandParameter="{Binding SelectedPlaylist}"
                    BackgroundColor="#D32F2F"
                    TextColor="White" />
        </Grid>

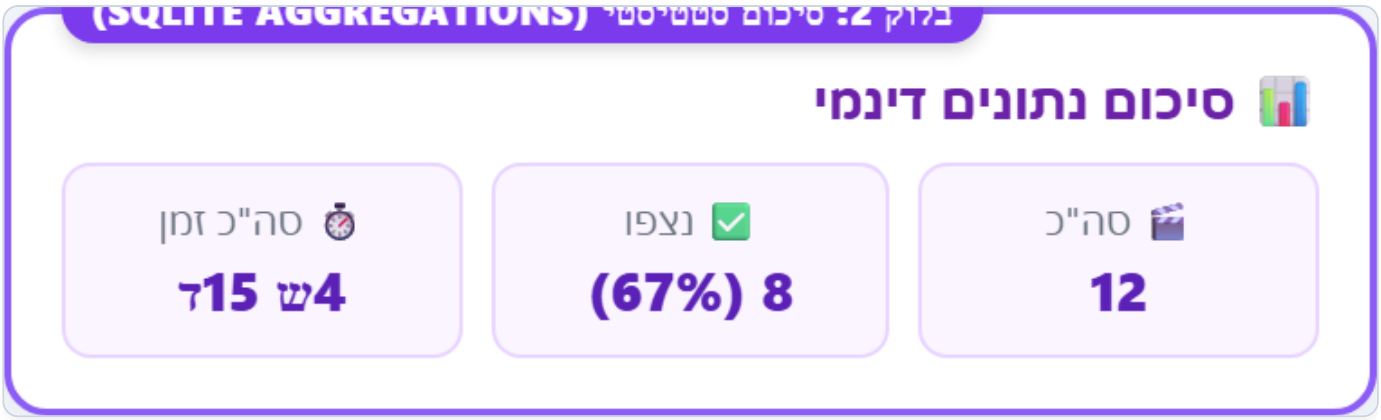
        <!-- קו מפריד וטופס יצירת פלייליסט חדש -->
        <BoxView HeightRequest="1" Color="#CCCCCC" Margin="0,4" />
        <Label Text="➕ יצירת פלייליסט חדש" FontSize="15" FontAttributes="Bold" />

        <Grid RowDefinitions="Auto, Auto"
              ColumnDefinitions="*, Auto"
              RowSpacing="8"
              ColumnSpacing="10">
            <Entry Grid.Row="0"
                   Grid.Column="0"
                   Grid.ColumnSpan="2"
                   Placeholder="שם הפלייליסט החדש (חובה)"
                   Text="{Binding NewPlaylistName}" />

            <Entry Grid.Row="1"
                   Grid.Column="0"
                   Placeholder="תיאור הפלייליסט (אופציונלי)"
                   Text="{Binding NewPlaylistDescription}" />

            <Button Grid.Row="1"
                    Grid.Column="1"
                    Text="צור פלייליסט"
                    Command="{Binding AddPlaylistCommand}"
                    BackgroundColor="#2E7D32"
                    TextColor="White" />
        </Grid>
    </VerticalStackLayout>
</Border>
```

שלב 4.3: בלוק 2 — סיכום סטטיסטי מחושב (SQLite Aggregations)



בלוק זה מוצג רק כאשר נבחר פלייליסט (`IsVisible="{Binding HasStats}"`). הוא מציג שלושה כרטיסי סיכום מעוצבים בתוך גריד של 3 עמודות שוות:

XML

```xml
<!-- חלק 2: סיכום סטטיסטי של הפלייליסט (SQLite Aggregations) -->
<Border StrokeShape="RoundRectangle 10"
        Padding="14"
        BackgroundColor="{AppThemeBinding Light=#EDE7F6, Dark=#311B92}"
        IsVisible="{Binding HasStats}">
    <VerticalStackLayout Spacing="8">
        <Label Text="📊 סיכום סטטיסטי (SQLite Aggregations)"
               FontSize="15"
               FontAttributes="Bold"
               TextColor="{AppThemeBinding Light=#4A148C, Dark=#E1BEE7}" />

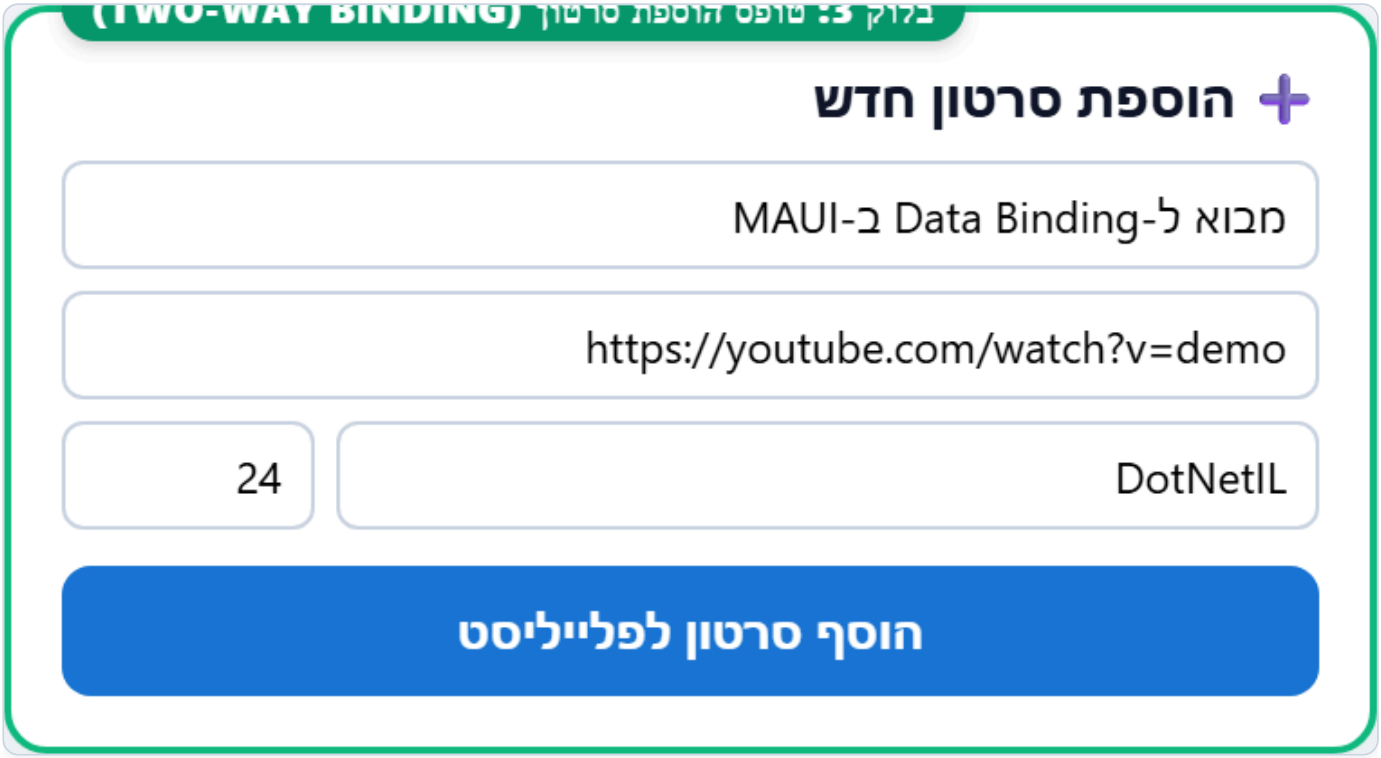
        <Grid ColumnDefinitions="*, *, *" ColumnSpacing="8">
            <!-- כרטיס 1: סה"כ סרטונים -->
            <Border StrokeShape="RoundRectangle 8"
                    BackgroundColor="{AppThemeBinding Light=White, Dark=#4527A0}"
                    Padding="8">
                <VerticalStackLayout HorizontalOptions="Center" Spacing="2">
                    <Label Text="🎬 סה&amp;quot;כ"
                           FontSize="11"
                           TextColor="Gray"
                           HorizontalOptions="Center" />
                    <Label Text="{Binding CurrentPlaylistStats.TotalVideos}"
                           FontSize="18"
                           FontAttributes="Bold"
                           HorizontalOptions="Center" />
                </VerticalStackLayout>
            </Border>

            <!-- כרטיס 2: סרטונים שנצפו ואחוז השלמה -->
            <Border StrokeShape="RoundRectangle 8"
                    BackgroundColor="{AppThemeBinding Light=White, Dark=#4527A0}"
                    Padding="8">
                <VerticalStackLayout HorizontalOptions="Center" Spacing="2">
                    <Label Text="✅ נצפו"
                           FontSize="11"
                           TextColor="Gray"
                           HorizontalOptions="Center" />
                    <Label Text="{Binding CurrentPlaylistStats.WatchedVideos, StringFormat='{0}
סרטונים'}"
                           FontSize="13"
                           FontAttributes="Bold"
                           HorizontalOptions="Center" />
                    <Label Text="{Binding CurrentPlaylistStats.CompletionPercentage,
StringFormat='({0}%)'}"
                           FontSize="11"
                           TextColor="#2E7D32"
                           HorizontalOptions="Center" />
                </VerticalStackLayout>
            </Border>

            <!-- כרטיס 3: סך דקות צפייה -->
            <Border StrokeShape="RoundRectangle 8"
                    BackgroundColor="{AppThemeBinding Light=White, Dark=#4527A0}"
                    Padding="8">
                <VerticalStackLayout HorizontalOptions="Center" Spacing="2">
                    <Label Text="⏱ סה&amp;quot;כ זמן"
                           FontSize="11"
```

```
                        TextColor="Gray"
                        HorizontalOptions="Center" />
                <Label Text="{Binding CurrentPlaylistStats.FormattedTotalDuration}"
                        FontSize="13"
                        FontAttributes="Bold"
                        HorizontalOptions="Center" />
            </VerticalStackLayout>
        </Border>
    </Grid>
</VerticalStackLayout>
</Border>
```

שלב 4.4: בלוק 3 — טופס הוספת סרטון לפלייליסט (Add Video Form)



בלוק זה מכיל שדות קלט מותאמים עם סוג מקלדת ייעודי (`Keyboard="Url"` ו-`Keyboard="Numeric"`):

XML

```xml
<!-- חלק 3: טופס הוספת סרטון לפלייליסט הנבחר -->
<Border StrokeShape="RoundRectangle 10"
        Padding="16"
        BackgroundColor="{AppThemeBinding Light=#FAFAFA, Dark=#1E1E1E}"
        IsVisible="{Binding HasSelectedPlaylist}">
    <VerticalStackLayout Spacing="10">
        <Label Text="➕ הוספת סרטון לפלייליסט הנוכחי"
               FontSize="18"
               FontAttributes="Bold" />

        <Entry Placeholder="כותרת הסרטון (חובה)"
               Text="{Binding NewVideoTitle}" />

        <Entry Placeholder="קישור URL לסרטון ב-YouTube (חובה)"
               Text="{Binding NewVideoUrl}"
               Keyboard="Url" />

        <Grid ColumnDefinitions="*, *" ColumnSpacing="10">
            <Entry Grid.Column="0"
                   Placeholder="שם הערוץ"
                   Text="{Binding NewVideoChannel}" />

            <Entry Grid.Column="1"
                   Placeholder="משך בדקות"
                   Text="{Binding NewVideoDuration}"
                   Keyboard="Numeric" />
        </Grid>

        <Entry Placeholder="הערות אישיות על הסרטון"
               Text="{Binding NewVideoNotes}" />

        <Button Text="הוסף סרטון לפלייליסט"
                Command="{Binding AddVideoCommand}"
                BackgroundColor="#1976D2"
                TextColor="White"
                FontAttributes="Bold"
                Margin="0,6,0,0" />
    </VerticalStackLayout>
</Border>
```

שלב 4.5: בלוק 4 — סרגל חיפוש, מיון וסינון ב-SQL (Filter Bar)



בלוק זה משלב שדה חיפוש בזמן אמת, תיבת סימון (CheckBox) לסינון פריטים שלא נצפו, ותיבת בחירה (Picker) למיון:

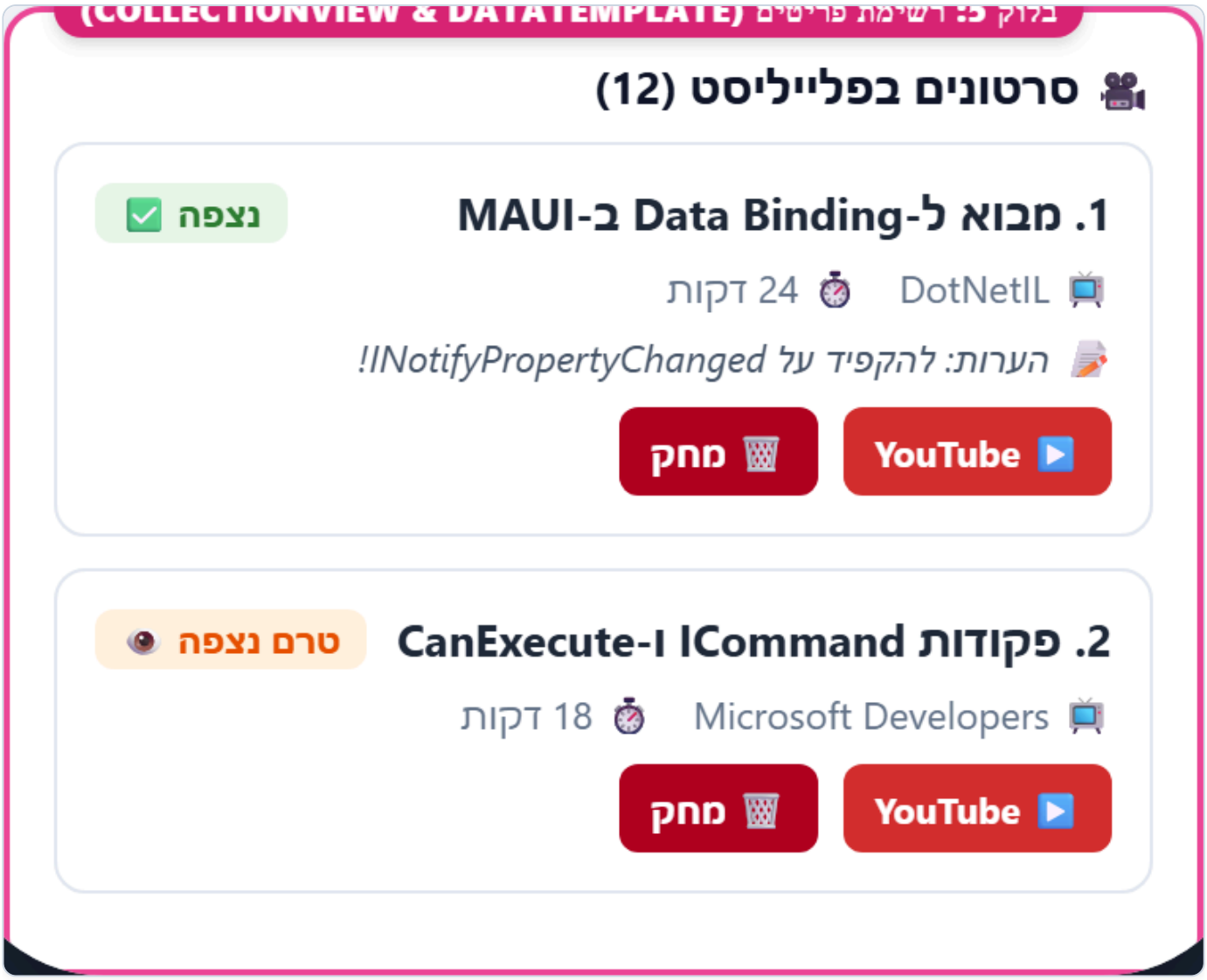
XML

```xml
<!-- חלק 4: סרגל חיפוש, סינון ומיון ב-SQL -->
<Border StrokeShape="RoundRectangle 10"
        Padding="14"
        BackgroundColor="{AppThemeBinding Light=#ECEFF1, Dark=#263238}"
        IsVisible="{Binding HasSelectedPlaylist}">
    <VerticalStackLayout Spacing="10">
        <Entry Placeholder="🔍 חפש לפי כותרת, ערוץ או הערות..."
               Text="{Binding SearchQuery}" />

        <Grid ColumnDefinitions="*, Auto"
              ColumnSpacing="12"
              VerticalOptions="Center">
            <!-- בורר אפשרויות מיון -->
            <HorizontalStackLayout Grid.Column="0"
                                   Spacing="6"
                                   VerticalOptions="Center">
                <Label Text="מיון:"
                       VerticalOptions="Center"
                       FontSize="13"
                       TextColor="Gray" />
                <Picker Title="בחר מיון..."
                        ItemsSource="{Binding AvailableSortOptions}"
                        ItemDisplayBinding="{Binding DisplayName}"
                        SelectedItem="{Binding SelectedSortOptionItem}"
                        FontSize="13"
                        HorizontalOptions="Fill" />
            </HorizontalStackLayout>

            <!-- סינון רק שלא נצפו -->
            <HorizontalStackLayout Grid.Column="1"
                                   Spacing="4"
                                   VerticalOptions="Center">
                <CheckBox IsChecked="{Binding ShowOnlyUnwatched}"
                          VerticalOptions="Center" />
                <Label Text="רק שלא נצפו"
                       VerticalOptions="Center"
                       FontSize="13" />
            </HorizontalStackLayout>
        </Grid>
    </VerticalStackLayout>
</Border>
```

שלב 4.6: בלוק 5 — רשימת כרטיסים דינמית (CollectionView & DataTemplate)



בלוק זה מציג את כרטיסי הסרטונים. כל כרטיס כולל `DataTrigger` לשינוי מצב כפתור הצפייה וכפתור לפתיחת הסרטון באפליקציית YouTube החיצונית:

XML

```xml
<!-- חלק 5: רשימת סרטונים (CollectionView) -->
<VerticalStackLayout Spacing="10" IsVisible="{Binding HasSelectedPlaylist}">
    <Label Text="🎥 סרטונים בפלייליסט" FontSize="18" FontAttributes="Bold" />

    <CollectionView ItemsSource="{Binding Videos}" SelectionMode="None">
        <!-- תצוגה כאשר הרשימה ריקה -->
        <CollectionView.EmptyView>
            <VerticalStackLayout Padding="30" HorizontalOptions="Center" Spacing="10">
                <Label Text="📭 אין סרטונים להצגה בפלייליסט זה" FontSize="16" HorizontalOptions="Center"
TextColor="Gray" />
                <Label Text="השתמש בטופס למעלה כדי להוסיף סרטונים חדשים!" FontSize="13"
HorizontalOptions="Center" TextColor="Gray" />
            </VerticalStackLayout>
        </CollectionView.EmptyView>

        <!-- תבנית הפריט הבודד -->
        <CollectionView.ItemTemplate>
            <DataTemplate x:DataType="models:VideoItem">
                <Border StrokeShape="RoundRectangle 8"
                        Margin="0,0,0,10"
                        Padding="14"
                        BackgroundColor="{AppThemeBinding Light=White, Dark=#2D2D2D}"
                        StrokeThickness="1"
                        Stroke="#E0E0E0">
                    <Grid RowDefinitions="Auto, Auto, Auto" ColumnDefinitions="*, Auto"
RowSpacing="6">

                        <!-- כותרת, ערוץ וזמן -->
                        <VerticalStackLayout Grid.Row="0" Grid.Column="0" Spacing="2">
                            <Label Text="{Binding Title}"
                                   FontSize="16"
                                   FontAttributes="Bold"
                                   LineBreakMode="TailTruncation" />
                            <HorizontalStackLayout Spacing="12">
                                <Label Text="{Binding ChannelName, StringFormat='📺 {0} :ערוץ'}"
                                       FontSize="13"
                                       TextColor="Gray" />
                                <Label Text="{Binding FormattedDuration, StringFormat='⏱ {0}'}"
                                       FontSize="13"
                                       TextColor="Gray" />
                            </HorizontalStackLayout>
                        </VerticalStackLayout>

                        <!-- כפתור מצב צפייה עם DataTrigger -->
                        <Button Grid.Row="0"
                                Grid.Column="1"
                                Text="טרם נצפה 👁"
                                Command="{Binding BindingContext.ToggleWatchedCommand,
                                                  Source={RelativeSource AncestorType={x:Type
ContentPage}}}"
                                CommandParameter="{Binding .}"
                                FontSize="12"
                                HeightRequest="36"
                                Padding="8,2"
                                BackgroundColor="#FFF3E0"
                                TextColor="#E65100">
                            <Button.Triggers>
```

```
                            <DataTrigger TargetType="Button"
                                         Binding="{Binding IsWatched}"
                                         Value="True">
                                <Setter Property="Text" Value="נצפה ✅" />
                                <Setter Property="BackgroundColor" Value="#E8F5E9" />
                                <Setter Property="TextColor" Value="#2E7D32" />
                            </DataTrigger>
                        </Button.Triggers>
                    </Button>

                    <!-- הערות אישיות -->
                    <Label Grid.Row="1"
                           Grid.Column="0"
                           Grid.ColumnSpan="2"
                           Text="{Binding Notes, StringFormat='📝 {0} :הערות'}"
                           FontSize="13"
                           FontAttributes="Italic"
                           TextColor="#555555" />

                    <!-- כפתורי פעולה: נגן ומחק -->
                    <HorizontalStackLayout Grid.Row="2"
                                           Grid.Column="0"
                                           Grid.ColumnSpan="2"
                                           Spacing="10"
                                           Margin="0,6,0,0">
                        <Button Text="▶️ צפה ב-YouTube (Launcher)"
                                Command="{Binding BindingContext.OpenVideoUrlCommand,
                                                  Source={RelativeSource AncestorType=
{x:Type ContentPage}}}"
                                CommandParameter="{Binding .}"
                                BackgroundColor="#FF0000"
                                TextColor="White"
                                FontSize="13"
                                HeightRequest="36"
                                Padding="12,0" />

                        <Button Text="🗑 מחק סרטון"
                                Command="{Binding BindingContext.DeleteVideoCommand,
                                                  Source={RelativeSource AncestorType=
{x:Type ContentPage}}}"
                                CommandParameter="{Binding .}"
                                BackgroundColor="#B00020"
                                TextColor="White"
                                FontSize="13"
                                HeightRequest="36"
                                Padding="12,0" />
                    </HorizontalStackLayout>

                </Grid>
            </Border>
        </DataTemplate>
    </CollectionView.ItemTemplate>
</CollectionView>
</VerticalStackLayout>
```

✍️ 5. פינות העמקה והבנה מושגית (Deep-Dive Callouts)

🔍 1. למה משתמשים ב-`Border` עם `StrokeShape` במקום ב-`Frame`?

בגרסאות ישנות של Xamarin.Forms השתמשו ב-`Frame`. ב-.NET MAUI, הרכיב המומלץ והביצועי ביותר הוא `Border`. הוא מאפשר שליטה מדויקת ברדיוס הפינות בעזרת `RoundRectangle`, מסגרת נקייה (`StrokeThickness="0"`), וביצועי רינדור מהירים בהרבה ללא שכבות מיותרות!

🔗 תיעוד רשמי: Microsoft Learn - .NET MAUI Border

🔍 2. איך כפתור בתוך `DataTemplate` מפעיל פקודה ב-ViewModel של הדף?

בתוך `ItemTemplate`, ה-BindingContext של הפריט הוא אובייקט מסוג `VideoItem` (ולא ה-`MainViewModel`). לכן, אם נכתוב סתם `Command="{Binding DeleteVideoCommand}"`, התוכנית לא תמצא את הפקודה! הפתרון הוא להשתמש ב-`RelativeSource`:

```xml
Command="{Binding BindingContext.DeleteVideoCommand, Source={RelativeSource
AncestorType={x:Type ContentPage}}}"
CommandParameter="{Binding .}"
```

הוראה זו מורה ל-MAUI: *"טפס במעלה עץ הפקדים עד שתגיע ל-ContentPage, קח את ה-BindingContext שלו (ה-ViewModel), והפעל את הפקודה עם הפריט הנוכחי (`Binding .`)"*.

🔗 תיעוד רשמי: Microsoft Learn - Relative Bindings in MAUI

🔍 3. התאמה אוטומטית למצב בהיר וכהה (`AppThemeBinding`)

שימו לב לקוד: `BackgroundColor="{AppThemeBinding Light=#F5F5F5, Dark=#212121}"`. בעזרת מנגנון זה, האפליקציה מחליפה צבעים אוטומטית כשהמשתמש משנה את מצב המכשיר שלו בין Dark Mode ל-Light Mode — ללא צורך בכתיבת קוד #C נוסף!

🔗 תיעוד רשמי: Microsoft Learn - App Theme Binding

🔌 6. ארכיטקטורת MVVM וחיבור הנתונים (ViewModel & Commands)

כדי לתמוך בכל העושר הוויזואלי שבנינו, ה-`MainViewModel` מנהל את המצב בצורה נקייה וסדורה:

1. ניהול שדות ותכונות עם `OnPropertyChanged`

כל תכונה מקושרת לשדה פרטי (`backing field`), ומעדכנת את התצוגה רק בעת שינוי ערך אמיתי:

CSHARP

```
private Playlist? _selectedPlaylist;
public Playlist? SelectedPlaylist
{
    get => _selectedPlaylist;
    set
    {
        if (_selectedPlaylist != value)
        {
            _selectedPlaylist = value;
            OnPropertyChanged();
            OnPropertyChanged(nameof(HasSelectedPlaylist));
            _ = OnSelectedPlaylistChangedAsync();
        }
    }
}
```

2. עקרון פיתוח מקצועי: הגנה על פקודות באמצעות `CanExecute`

במקום להדליק ולכבות כפתורים בצורה ידנית ב-Code-Behind, אנו מגדירים תנאי ביצוע ישירות בפקודה:

CSHARP

```
AddVideoCommand = new Command(
    async () => await AddVideoAsync(),
    () => HasSelectedPlaylist && !IsLoading && !string.IsNullOrWhiteSpace(NewVideoTitle)
);
```

כאשר משתנה הסטטוס (`HasSelectedPlaylist` או `IsLoading`), אנו מפעילים:

CSHARP

```
((Command)AddVideoCommand).ChangeCanExecute();
```

כך ה-Button ב-XAML מתעדכן ומנטרל את עצמו אוטומטית!

3. שימוש ב-Launcher לפתיחת קישורים חיצוניים

כאשר המשתמש לוחץ על כפתור ה-YouTube, ה-ViewModel מתקשר עם מערכת ההפעלה באופן בטוח:

CSHARP

```csharp
private async Task OpenVideoUrlAsync(VideoItem? video)
{
    if (video == null || string.IsNullOrWhiteSpace(video.Url)) return;

    if (Uri.TryCreate(video.Url, UriKind.Absolute, out var uri))
    {
        await Launcher.Default.OpenAsync(uri);
    }
}
```

❓ 7. שאלות חזרה, הבנה ואתגר חשיבה

⚖️ חלק א': דילמת ארכיטקטורה ועיצוב (Architectural Dilemma)

הדילמה: איפה לבצע חישובי סיכום — במסד הנתונים (SQL) או בזיכרון (C#)?
במסך שלנו יצרנו כרטיסי סיכום סטטיסטיים (סה"כ סרטונים, נצפו, וסך דקות).

- **גישה א':** לשלוף את כל רשימת הסרטונים ל-C# ולהפעיל `Videos.Where(v ⇒ v.IsWatched).Count()`, `Videos.Count()` ו-`Videos.Sum(v ⇒ v.DurationMinutes)`.
- **גישה ב':** לשלוח שאילתת אגרגציה ממוקדת ל-SQLite (`SELECT COUNT(*), SUM(DurationMinutes) ...`) והמחזירה רק את הערכים המספריים.

> 💡 **שיקולי החלטה למפתח:**
>
> - **ביצועים:** כאשר בפלייליסט יש מאות או אלפי סרטונים, גישה א' טוענת אובייקטים כבדים לזיכרון ה-RAM ומאטה את המכשיר. גישה ב' מנצלת את מנוע ה-C של SQLite ומחזירה תשובה במיקרו-שנייה.
> - **תזמון:** גישה ב' עדיפה תמיד כשאנו זקוקים לסיכום בלבד ללא הצגת כל הפריטים.

🧠 חלק ב': שאלות הבנה מעמיקות עם רמזי הכוונה

1. **מדוע הגדרנו שורה בגובה `*` עבור אזור הגלילה הראשי (`ScrollView`) במקום לקבוע גובה קבוע במספרים (למשל `HeightRequest="600"`)?**

> 💡 **רמז לחשיבה:** חשבו על מכשירי סמארטפון בעלי גדלי מסך שונים — ממסך 5.5 אינץ' ועד טאבלט 12 אינץ'. מה קורה כאשר מגדירים גובה קשיח?

2. **מה תפקידו של `RelativeSource AncestorType={x:Type ContentPage}` בכפתור המחיקה שבתוך ה-`DataTemplate`?**

> 💡 **רמז לחשיבה:** מי ה-`BindingContext` של השורה הבודדת בתוך הרשימה? האם לפריט בודד מסוג `VideoItem` יש פקודת `DeleteVideoCommand`?

3. **כיצד מנגנון ה-`AppThemeBinding` עוזר לשמור על חוויית משתמש נגישה במצב כהה (Dark Mode)?**

> 💡 **רמז לחשיבה:** נסו לחשוב מה קורה לטקסט שחור כאשר רקע הכרטיס מוגדר קשיח כ-`White` במכשיר המוגדר כ-Dark Theme.

🛠️ חלק ג': אתגר מעשי קצר לתלמיד (Mini-Challenge)

המשימה:

הוסיפו כרטיס סטטיסטי רביעי בבלוק 2 המציג את **זמן הצפייה הממוצע** לסרטון בפלייליסט (בדקות).

שלבי יישום מודרכים:

1. **שכבת ה-Model / DTO (`PlaylistStats.cs`):** הוסיפו תכונה מחושבת:

CSHARP

```
public double AverageDurationMinutes ⇒
    TotalVideos > 0 ? Math.Round((double)TotalDurationMinutes / TotalVideos, 1) : 0;
```

2. **שכבת ה-XAML (`MainPage.xaml`):** שנו את הגריד של בלוק 2 מ-`ColumnDefinitions="*, *, *"` ל-`ColumnDefinitions="*, *, *, *"`, והוסיפו עמודה רביעית:

XML

```
<!-- כרטיס 4: ממוצע דקות לסרטון -->
 <Border Grid.Column="3"
         StrokeShape="RoundRectangle 8"
         BackgroundColor="{AppThemeBinding Light=White, Dark=#4527A0}"
         Padding="8">
     <VerticalStackLayout HorizontalOptions="Center" Spacing="2">
         <Label Text="📈 ממוצע"
                FontSize="11"
                TextColor="Gray"
                HorizontalOptions="Center" />
         <Label Text="{Binding CurrentPlaylistStats.AverageDurationMinutes, StringFormat='{0}
דק'''}"
                FontSize="13"
                FontAttributes="Bold"
                HorizontalOptions="Center" />
     </VerticalStackLayout>
 </Border>
```

3. **אימות:** הריצו את האפליקציה, בחרו פלייליסט וודאו שהכרטיס הרביעי מציג את הממוצע המחושב בצורה תקינה.

📖 נספח: מקורות השראה, תיעוד וביבליוגרפיה

מדריך זה נבנה בהשראת מתודולוגיית הפירוק המודולרי של ממשקי מובייל (The Block Strategy) כפי שפותחה ע"י **Leomaris Reyes** (Microsoft MVP, יוצרת AskXammy.com וכותבת סדרת מאמרי ממשק ב-Telerik).

תיעוד רשמי ומקורות נוספים להעמקה:

- Microsoft Learn: מדריך הפיתוח הרשמי ל-.NET MAUI
- Microsoft Learn: עקרונות Data Binding וארכיטקטורת MVVM
- Microsoft Learn: שימוש ברכיב ה-Border ועיגול פינות עם StrokeShape
- Microsoft Learn: עבודה עם מסדי נתונים מקומיים ב-SQLite